

Linear algebra for machine learning

A practical guide to vectors, matrices, dot products, norms, projections, linear regression and loss functions. Build the intuition first, then calculate by hand, then connect the result to model training.

Level: foundations through introductory ML. Basic algebra is enough for most of the guide; the gradient section introduces the derivative you need. All vectors and matrices here are real-valued.

The central connection

Features → weighted predictions → residuals → loss → better weights

A vector stores one example. A matrix stores many examples. A dot product makes a weighted prediction. A norm measures the size of the errors. Least-squares regression chooses the weights that make those errors as small as possible. Projection explains the geometry of that choice.

How to use this guide

Pass	What to do
1 · Understand	Read pages 2-4. Recalculate each short example without looking at the result.
2 · Connect	Work through the complete regression fit on page 5, then compare losses and training on pages 6-7.
3 · Test	Answer the three questions on page 8 before checking pages 9-10. Use the source links on page 11 for deeper study.

Notation you will see

n: number of examples; **d**: number of features; **x**: one feature vector; **X**: data matrix; **w**: feature weights; **b**: intercept; **y**: observed targets; **$\hat{y}$** : predictions. A superscript T means transpose, not a power.

Source note: This is an independently written study guide aligned with the MIT resources listed on page 11, not an official MIT publication. Worked examples and self-test questions are original, not MIT exam questions. The loss comparison also includes standard supplementary ML material.

Vectors, matrices and dimensions

A vector: one example, several features

A vector is an ordered list of numbers. For a toy prediction problem, $x = [2, 3]^T$ could contain two input measurements. The order matters: swapping features changes their meaning. We treat x as a column vector with shape $d \times 1$.

$$x = [2, 3]^T, z = [1, -1]^T$$

$$x + z = [3, 2]^T, 2x = [4, 6]^T$$

Vector addition combines corresponding coordinates; scalar multiplication scales every coordinate. A linear combination such as $2x - z$ combines scaling and addition. The span of vectors is the set of all their linear combinations.

A matrix: a dataset or a transformation

Use one row per example and one column per feature. Other sources sometimes put examples in columns, so always check the convention.

$$X = \begin{bmatrix} 2 & 3 \\ 4 & 1 \\ 0 & 2 \end{bmatrix}, w = [1, 2]^T$$

$$Xw = [8, 6, 4]^T$$

The first output is $2 \times 1 + 3 \times 2 = 8$. The remaining outputs are calculated in the same way. Xw also equals the first column of X plus twice the second column: both interpretations describe the same multiplication.

Object	Shape	Meaning
X	$n \times d$	All input examples
w	$d \times 1$	One weight per feature
Xw	$n \times 1$	One prediction per example
x^T	$d \times n$	Rows and columns exchanged
$x^T x$	$d \times d$	Feature-column inner products

Rule: $(m \times k)(k \times r)$ gives an $m \times r$ matrix. Inner dimensions must agree. Matrix multiplication is generally not commutative: AB and BA may differ, or one may not exist.

ML pitfall: elementwise multiplication and matrix multiplication are different operations. In NumPy, $X @ w$ forms the weighted sums; $X * w$ usually uses broadcasting to multiply entries.

Dot products and norms

Dot product: a weighted sum and an alignment measure

$$x^T w = \sum_{j=1}^d x_j w_j$$

$$x^T w = \|x\|_2 \|w\|_2 \cos(\theta)$$

Multiply matching coordinates and add. The result is one number, not a vector. Geometrically, θ is the angle between two nonzero vectors. A positive dot product means an acute angle; zero means orthogonal; negative means an obtuse angle.

Example: $x = [2, 3]^T$ and $w = [1, 2]^T$ give $x^T w = 8$. A linear model with intercept $b = -1$ predicts 7. With $v = [3, -2]^T$, $x^T v = 0$, so x and v are perpendicular.

Norm: the size of a vector

Norm	Formula	For $v = [3, -4]^T$
L1	$\ v\ _1 = \sum v_j $	7
L2 / Euclidean	$\ v\ _2 = \sqrt{\sum v_j^2}$	5
L-infinity	$\ v\ _\infty = \max v_j $	4

A norm is nonnegative, equals zero only at the zero vector, scales by the absolute value of a scalar, and satisfies the triangle inequality. Distance between x and z can be defined as $\|x - z\|$. Different norms encode different notions of distance.

Unit vector: $u = v / \|v\|_2$, when $v \neq 0$

Cosine similarity: $(x^T z) / (\|x\|_2 \|z\|_2)$

For $v = [3, -4]^T$, the unit vector is $[0.6, -0.8]^T$. It has the same direction and length 1. Cosine similarity compares direction while removing magnitude; it is undefined if either vector is zero.

ML connection: norms measure prediction errors and regularize weights. Large-scale features can dominate distances; standardize features when appropriate, fitting the scaler only on training data. Squared L2, $\|v\|_2^2$, is useful but is not itself a norm.

Projections: the closest allowed vector

Project onto one direction

Suppose you can only choose a multiple of a nonzero vector a . Which multiple lies closest to y ? Call the answer $p = ca$. The leftover error $e = y - p$ must be perpendicular to a ; otherwise, moving along a could reduce the distance further.

$$\begin{aligned} a^T(y - ca) &= 0 \\ c &= (a^T y) / (a^T a) \\ p &= a(a^T y) / (a^T a) \end{aligned}$$

Worked example: $y = [3, 1]^T$, $a = [1, 1]^T$. Then $c = (3 + 1) / (1 + 1) = 2$, so $p = [2, 2]^T$. The residual $e = [1, -1]^T$ satisfies $a^T e = 0$. Its length is $\sqrt{2}$.

The orthogonal pieces satisfy the Pythagorean identity: $\|y\|_2^2 = \|p\|_2^2 + \|e\|_2^2$. Here $10 = 8 + 2$. The coefficient c is not generally the length of p : that length is $|c| \|a\|_2$.

Project onto several directions

The column space of A is all vectors $A\beta$ that its columns can produce. Least squares finds the member of this space closest to y in Euclidean distance. The residual must be perpendicular to every column.

$$\begin{aligned} A^T(y - A\beta) &= 0 \\ A^T A \beta &= A^T y \\ \beta &= (A^T A)^{-1} A^T y \text{ [full column rank]} \end{aligned}$$

Full column rank means no column is a linear combination of the others. Under that condition, $A^T A$ is invertible. The projection matrix is $P = A(A^T A)^{-1} A^T$, so $\hat{y} = Py$; it satisfies $P^T = P$ and $P^2 = P$.

If columns are dependent: the best fitted vector $\hat{y}$ remains unique, but the coefficients need not be. The pseudoinverse gives one minimum-norm solution. In software, use a least-squares solver based on QR or SVD rather than explicitly forming the inverse.

Key distinction: this projection is in the space of n target values, not necessarily the original d -dimensional feature space.

Linear regression: a complete fit

Regression predicts a numeric target. For one example, $\hat{y} = w^T x + b$. The intercept b permits a nonzero prediction at $x = 0$. “Linear” refers to the parameters: using features such as x^2 can produce a curved function while remaining linear in its coefficients.

1. Build the design matrix

Fit the three toy observations $(x, y) = (0, 1), (1, 2), (2, 2)$. Add a column of ones to absorb the intercept into $\beta = [b, w]^T$.

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, y = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}^T$$

$$\hat{y} = A\beta, J(\beta) = (1/3)\|A\beta - y\|_2^2$$

2. Form and solve the normal equations

$$A^T A = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, A^T y = \begin{bmatrix} 5 \\ 6 \end{bmatrix}^T$$

$$3b + 3w = 5; 3b + 5w = 6$$

$$2w = 1 \Rightarrow w = 1/2; b = 7/6$$

The fitted model is $\hat{y} = 7/6 + x/2$. For a new input $x = 3$, it predicts $8/3$. This is extrapolation beyond the observed x range, so accuracy is not guaranteed.

3. Inspect the predictions and errors

$$\hat{y} = \begin{bmatrix} 7/6 \\ 5/3 \\ 13/6 \end{bmatrix}^T$$

$$e = y - \hat{y} = \begin{bmatrix} -1/6 \\ 1/3 \\ -1/6 \end{bmatrix}^T$$

$$SSE = 1/36 + 1/9 + 1/36 = 1/6$$

$$MSE = SSE/3 = 1/18 \approx 0.0556$$

Check the geometry: the residuals sum to zero, and their dot product with $[0, 1, 2]^T$ is zero. Thus $A^T e = 0$. The model cannot hit all three targets exactly, but no other line has smaller training squared error.

What this does not prove: a small training loss does not establish good future predictions, causality, or a correct linear model. Least squares can be computed without Gaussian noise; Gaussian, constant-variance errors provide an additional statistical justification, not a requirement for the calculation.

Loss functions: what counts as a bad error?

Let $r = \hat{y} - y$ be a prediction error. A per-example loss assigns a penalty to r ; the dataset objective usually averages these penalties. The previous page used $e = y - \hat{y}$: changing the sign does not affect absolute or squared errors.

Loss / metric	Definition	What it emphasizes
Squared loss / MSE	r^2 ; $\text{MSE} = (1/n)\sum r_i^2$	Large residuals receive strong penalties. Smooth derivative $2r$.
Absolute loss / MAE	$ r $; $\text{MAE} = (1/n)\sum r_i $	Linear growth reduces sensitivity to large target errors. Not differentiable at zero.
RMSE	$\sqrt{\text{MSE}}$	Same units as the target; same minimizers as MSE on a fixed dataset.
Huber loss	$r^2/2$ if $ r \leq \delta$; $\delta(r - \delta/2)$ otherwise	Quadratic near zero, linear far away; choose $\delta > 0$.

An error that grows from 2 to 10

Squared loss grows from 4 to 100: 25 times larger. Absolute loss grows from 2 to 10: 5 times larger. With Huber $\delta = 1$, the losses are 1.5 and 9.5. This illustrates why loss choice can change the fitted model.

For residuals $[1, -2, 3]$, $\text{MSE} = 14/3 \approx 4.667$, $\text{MAE} = 2$, and $\text{RMSE} \approx 2.160$. Never average signed residuals to measure accuracy: positive and negative errors can cancel.

Classification uses a different target

Binary cross-entropy = $-[y \ln(p) + (1 - y) \ln(1 - p)]$
 $y \in \{0, 1\}$, $0 < p < 1$

Here p is a predicted probability, not an unrestricted regression output. For $y = 1$, $p = 0.8$ gives loss ≈ 0.223 ; $p = 0.2$ gives ≈ 1.609 . A confident wrong prediction costs more. Numerically stable implementations work directly with logits.

Choosing a loss: match the target and the cost of mistakes. MSE suits strong penalties on large errors; MAE or Huber may help with large target outliers, but do not eliminate every kind of outlier problem. For unrestricted prediction functions, expected squared loss targets the conditional mean and absolute loss targets a conditional median. Select using validation data and the application goal.

Learning weights and avoiding misleading results

Gradient descent

A derivative measures how the loss changes when a parameter changes. The gradient collects those derivatives into a vector. Move against it to reduce the loss locally.

$$J(\beta) = (1/n) \|A\beta - y\|_2^2$$

$$\nabla J(\beta) = (2/n) A^T (A\beta - y)$$

$$\beta_{\text{new}} = \beta - \eta \nabla J(\beta), \eta > 0$$

Why this gradient? Each squared residual contributes $2r_i$; its derivative with respect to β is the corresponding row of A . Add these contributions and divide by n . Some texts use $1/(2n)$, which removes the leading 2 without changing the minimizer.

Using page 5, start at $\beta = [0, 0]^T$. Then $\nabla J = [-10/3, -4]^T$. With learning rate $\eta = 0.1$, the next β is $[1/3, 0.4]^T$. The MSE falls from 3 to about 0.954. A step that is too large can increase the loss or diverge.

Regularization uses norms of weights

$$\text{Ridge: } J = \text{MSE} + \lambda \|w\|_2^2$$

$$\text{Lasso: } J = \text{MSE} + \lambda \|w\|_1, \lambda \geq 0$$

Ridge discourages large weights; lasso can produce exactly zero coefficients. Typically the intercept is not penalized. Feature scale changes the effect of these penalties, so preprocessing matters. Tune λ on validation data, not the final test set. A larger penalty can underfit; regularization is not a guarantee of better results.

A minimal implementation of the worked fit

```
import numpy as np
x = np.array([0., 1., 2.])
y = np.array([1., 2., 2.])
A = np.column_stack([np.ones_like(x), x])
beta = np.linalg.lstsq(A, y, rcond=None)[0]
pred = A @ beta
mse = np.mean((pred - y)**2)
```

Evaluation checklist: keep an untouched test set; fit preprocessing only on training data; compare with a simple baseline; inspect residuals. For forecasting, use chronological splits and features available at prediction time. Randomly mixing future and past can give an unrealistic estimate of performance.

Self-test: three questions

Try these without the answer key. Show dimensions, intermediate steps and one sentence interpreting each result. Suggested time: 25-35 minutes. These are original practice questions.

Question 1 · Vectors, norms and projection (10 points)

Let $x = [3, 4]^T$, $w = [2, -1]^T$, $b = 1$ and $a = [1, 1]^T$.

- (a) Compute $x^T w$ and the prediction $w^T x + b$. (2)
- (b) Find $\|x\|_1$, $\|x\|_2$ and the L2-normalized x . (3)
- (c) Project x onto $\text{span}(a)$. Give the projection p and residual $e = x - p$, and verify orthogonality. (4)
- (d) Is $x^T w$ a scalar or a vector? (1)

Question 2 · Fit and interpret a regression (12 points)

Use the dataset $(x, y) = (0, 1), (1, 3), (2, 3)$. Fit $\hat{y} = b + wx$ by ordinary least squares.

- (a) Write A , y and β , including their shapes. (3)
- (b) Calculate $A^T A$ and $A^T y$; solve for b and w . (4)
- (c) Calculate predictions, residuals and training MSE. (3)
- (d) Explain why $A^T(y - A\beta) = 0$ and why this is not proof of good test performance. (2)

Question 3 · Loss choice and a training step (8 points)

- (a) Model A has residuals $[0, 0, 6]$; model B has residuals $[3, 3, 3]$, on the same three examples. Compute MAE and MSE for each. Which does each metric prefer, and why? (4)
- (b) For the one-parameter model $\hat{y} = wx$, take $x = [1, 2]^T$, $y = [2, 4]^T$, $w = 0$, and MSE as the objective. Compute dJ/dw and one gradient-descent update with $\eta = 0.1$. (3)
- (c) Should you choose your loss or learning rate by repeatedly testing on the final test set? Explain. (1)

Answer key · Questions 1 and 2

Question 1

(a) $x^T w = 3 \times 2 + 4 \times (-1) = 2$. Including the intercept gives $\hat{y} = 3$.

(b) $\|x\|_1 = 7$; $\|x\|_2 = \sqrt{9 + 16} = 5$. The unit vector is $[3/5, 4/5]^T = [0.6, 0.8]^T$.

(c) $c = (a^T x) / (a^T a) = 7/2$

$p = [3.5, 3.5]^T$

$e = [-0.5, 0.5]^T$; $a^T e = 0$

The residual has no component along the allowed direction, so the projection is the closest point on the line. **(d)** The dot product is a scalar: $(1 \times 2)(2 \times 1)$ gives 1×1 .

Question 2

(a) $A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$ has shape 3×2 ; $y = [1, 3, 3]^T$ has shape 3×1 ; $\beta = [b, w]^T$ has shape 2×1 .

(b) $A^T A = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$

$A^T y = [7, 9]^T$

$3b + 3w = 7$; $3b + 5w = 9$

$w = 1$; $b = 4/3$

(c) $\hat{y} = [4/3, 7/3, 10/3]^T$

$e = [-1/3, 2/3, -1/3]^T$

$SSE = 1/9 + 4/9 + 1/9 = 2/3$

$MSE = (2/3)/3 = 2/9 \approx 0.2222$

(d) The residual is perpendicular to both design columns: $-1/3 + 2/3 - 1/3 = 0$ and $0 + 2/3 - 2/3 = 0$. This is the first-order optimality condition for least squares and expresses projection onto the column space. It concerns the training data only; unseen data may have different patterns or expose poor model assumptions.

Answer key · Question 3 and review

Question 3

Model	MAE	MSE
A: [0, 0, 6]	$6/3 = 2$	$36/3 = 12$
B: [3, 3, 3]	$9/3 = 3$	$27/3 = 9$

(a) MAE prefers A; MSE prefers B. A has smaller average absolute error but one large mistake. Squaring makes that mistake costly enough to reverse the ranking. Neither choice is universally correct: decide which error pattern matters in the application.

(b) $J(w) = [(w - 2)^2 + (2w - 4)^2]/2$
 $dJ/dw = (w - 2) + 2(2w - 4) = 5w - 10$
 At $w = 0$: gradient = -10
 $w_{\text{new}} = 0 - 0.1(-10) = 1$

At $w = 0$, $MSE = (4 + 16)/2 = 10$. After the update, predictions are $[1, 2]^T$ and $MSE = (1 + 4)/2 = 2.5$. The optimum is $w = 2$, so one update improves the fit but does not finish training.

(c) No. Repeated test-based choices leak information about the test set into model selection. Use validation data for these choices, then evaluate the selected model on the untouched test set.

Use your mistakes to decide what to revisit

If you missed...	Review...
Shapes or weighted sums	Page 2: each prediction is a row-column dot product.
Normalization or projection	Pages 3-4: normalize by a norm; project with a dot-product ratio.
Regression coefficients	Page 5: add the intercept column, then solve the normal equations.
Loss rankings or gradient signs	Pages 6-7: write the objective explicitly before differentiating.

Informal self-check: 26-30 points: move on to regularization and gradient descent practice. 20-25: redo missed calculations. Below 20: revisit the worked examples and explain each symbol aloud. This is a study aid, not a validated assessment.

Reliable, free resources

These links were checked on 17 September 2026. The listed notes and OCW materials are freely readable; no paid certificate is needed. Follow the order below rather than trying to finish every course.

1. Gilbert Strang · ZoomNotes for Linear Algebra

Start with Part 1 for vectors and matrix ideas, then Part 4 for orthogonality and least squares. Part 10 connects linear algebra to learning from data. Compact reference; use alongside worked examples.

[Open official MIT resource](#)

2. MIT 18.06SC · Least squares unit

Study Orthogonal Vectors and Subspaces, Projections onto Subspaces, and Projection Matrices and Least Squares. Includes links to teaching and practice materials. Skip determinants and eigenvalues for this first pass.

[Open official MIT resource](#)

3. MIT 18.06 · Lecture 16

Gilbert Strang explains why line fitting and projection are the same least-squares problem. Watch after page 5 and compare the lecture notation with A and β here.

[Open official MIT resource](#)

4. MIT 6.390 · Regression notes

Read sections 2.1-2.5, then 2.7-2.8: prediction, squared loss, analytical fitting, regularization and evaluation. Compare matrix orientations carefully; conventions can differ.

[Open official MIT resource](#)

5. MIT 6.390 · Gradient descent notes

Continue after page 7. Focus on gradients, step size and how iterative optimization changes parameters. Basic differentiation helps.

[Open official MIT resource](#)

Attribution: MIT OpenCourseWare / Gilbert Strang and MIT 6.390 course staff. Referenced materials retain their own copyrights and licenses. No MIT endorsement is implied. Supplementary loss definitions and original exercises are included to connect the requested foundations to practical ML.